Pinning 고정

개요

Pinning(고정)은 IPFS에서 특정 콘텐츠가 가비지 컬렉션에 의해 삭제되지 않도록 보호하는 메커니즘입니다. Boxo의 pinning 모듈은 로컬 및 원격 고정 서비스를 제공하여 중요한 데이터를 안전하게 보관할 수 있게 해줍니다.

주요 개념

Pin 모드

IPFS에서는 다음과 같은 고정 모드를 지원합니다:

- **Recursive (재귀적)**: 대상 CID와 연결된 모든 하위 블록을 고정
- **Direct (직접적)**: 대상 CID만 고정
- **Indirect (간접적)**: 재귀적으로 고정된 블록의 하위 블록
- **Internal (내부적)**: Pinner의 내부 상태 유지용 블록

고정의 필요성

1. **데이터 보존**: 중요한 데이터가 가비지 컬렉션으로 삭제되는 것을 방지
2. **성능 향상**: 자주 사용하는 데이터를 로컬에 유지하여 빠른 접근 보장
3. **가용성 보장**: 네트워크에서 데이터가 사라지지 않도록 보장

기본 사용법

Pinner 인터페이스

```go
package main

import (
"context"
"fmt"
"log"

"github.com/ipfs/boxo/pinning/pinner"
"github.com/ipfs/boxo/pinning/pinner/dspinner"
"github.com/ipfs/boxo/ipld/merkledag"
"github.com/ipfs/go-cid"
"github.com/ipfs/go-datastore"
"github.com/ipfs/go-datastore/sync"
ipld  "github.com/ipfs/go-ipld-format"
)

func  main() {
ctx := context.Background()
// 데이터스토어와 DAG 서비스 설정
dstore := sync.MutexWrap(datastore.NewMapDatastore())
dagService := merkledag.NewDAGService(blockservice.New(blockstore.NewBlockstore(
```

```
// Pinner 생성

pinner, err := dspinner.New(ctx, dstore, dagService)

if err != nil {

log.Fatal(err)

}

// 노드 생성 및 고정

node := merkledag.NodeWithData([]byte("안녕하세요, IPFS!"))

// 재귀적 고정

err = pinner.Pin(ctx, node, true, "내 첫 번째 고정")

if err != nil {

log.Fatal(err)

}

// 고정 상태 확인

reason, pinned, err := pinner.IsPinned(ctx, node.Cid())

if err != nil {

log.Fatal(err)

}

if pinned {

fmt.Printf("CID %s가 고정되었습니다: %s\n", node.Cid(), reason)

}

}
```

고정 모드별 사용법

```
// 직접 고정 - CID만 고정
err = pinner.PinWithMode(ctx, cid, pin.Direct, "직접 고정")
if err != nil {
log.Fatal(err)
}

// 재귀적 고정 - CID와 모든 하위 블록 고정
err = pinner.PinWithMode(ctx, cid, pin.Recursive, "재귀적 고정")
if err != nil {
log.Fatal(err)
}

// 고정 해제
err = pinner.Unpin(ctx, cid, true) // true: 재귀적 고정 해제
if err != nil {
log.Fatal(err)
}
```

로컬 고정 서비스

고정 상태 조회

```
// 여러 CID의 고정 상태를 한 번에 확인
```

```
cids := []cid.Cid{cid1, cid2, cid3}

pinnedList, err := pinner.CheckIfPinned(ctx, cids...)

if err != nil {

log.Fatal(err)

}


for _, pinned := range pinnedList {

fmt.Printf("CID: %s, 상태: %s\n", pinned.Key, pinned.String())

}
```

고정된 항목 나열

```
// 직접 고정된 CID 목록

directPins := pinner.DirectKeys(ctx, true) // true: 상세 정보 포함

for streamedPin := range directPins {

if streamedPin.Err != nil {

log.Printf("오류: %v", streamedPin.Err)

continue

}

pin := streamedPin.Pin

fmt.Printf("직접 고정: %s (이름: %s)\n", pin.Key, pin.Name)

}
```

```
// 재귀적 고정된 CID 목록
recursivePins := pinner.RecursiveKeys(ctx, true)
for streamedPin := range recursivePins {
if streamedPin.Err != nil {
log.Printf("오류: %v", streamedPin.Err)
continue
}
pin := streamedPin.Pin
fmt.Printf("재귀적 고정: %s (이름: %s)\n", pin.Key, pin.Name)
}
```

고정 업데이트

```
// 고정을 한 CID에서 다른 CID로 업데이트
oldCid := cid.MustParse("QmOldContent...")
newCid := cid.MustParse("QmNewContent...")

err = pinner.Update(ctx, oldCid, newCid, true) // true: 이전 고정 해제
if err != nil {
log.Fatal(err)
}
```

```
fmt.Println("고정이 성공적으로 업데이트되었습니다")
```

원격 고정 서비스

원격 클라이언트 설정

```
package main

import (
"context"
"fmt"
"log"
"time"

"github.com/ipfs/boxo/pinning/remote/client"
"github.com/ipfs/go-cid"
)

func  main() {
ctx := context.Background()
// 원격 고정 서비스 클라이언트 생성
client := client.NewClient("https://api.pinata.cloud/psa", "your-bearer-token")
// CID 고정 요청
```

```
targetCid := cid.MustParse("QmYourContent...")

pinStatus, err := client.Add(ctx, targetCid,

client.PinOpts.WithName("중요한 문서"),

client.PinOpts.AddMeta(map[string]string{

"project": "my-project",

"type": "document",

}),

)

if  err != nil {

log.Fatal(err)

}

fmt.Printf("고정 요청 ID: %s\n", pinStatus.GetRequestId())

fmt.Printf("상태: %s\n", pinStatus.GetStatus())

}
```

고정 상태 모니터링

```
// 고정 상태 확인

pinID := "your-pin-request-id"

status, err := client.GetStatusByID(ctx, pinID)

if  err != nil {

log.Fatal(err)

}
```

```go
fmt.Printf("고정 상태: %s\n", status.GetStatus())
fmt.Printf("생성 시간: %s\n", status.GetCreated())


// 고정 완료까지 대기
for {
status, err := client.GetStatusByID(ctx, pinID)
if err != nil {
log.Fatal(err)
}
switch status.GetStatus() {
case client.StatusPinned:
fmt.Println("고정이 완료되었습니다!")
return
case client.StatusFailed:
fmt.Println("고정이 실패했습니다.")
return
default:
fmt.Printf("고정 진행 중... 상태: %s\n", status.GetStatus())
time.Sleep(5 * time.Second)
}
}
```

고정 목록 조회

```
// 모든 고정 목록 조회
pins, err := client.LsSync(ctx)
if err != nil {
log.Fatal(err)
}

for _, pin := range pins {
fmt.Printf("CID: %s, 이름: %s, 상태: %s\n",
pin.GetPin().GetCid(),
pin.GetPin().GetName(),
pin.GetStatus())
}

// 필터링된 고정 목록 조회
filteredPins, err := client.LsSync(ctx,
client.PinOpts.FilterStatus(client.StatusPinned),
client.PinOpts.FilterName("중요한 문서"),
client.PinOpts.Limit(50),
)
if err != nil {
log.Fatal(err)
```

```
}

fmt.Printf("필터링된 고정 개수: %d\n", len(filteredPins))
```

고급 활용

자동 동기화 설정

```
// 자동 동기화 비활성화 (대량 작업 시 성능 향상)
pinner.SetAutosync(false)

// 여러 고정 작업 수행
for _, cid := range largeCidList {
err := pinner.PinWithMode(ctx, cid, pin.Direct, "")
if err != nil {
log.Printf("고정 실패: %v", err)
}
}

// 수동으로 플러시
err = pinner.Flush(ctx)
if err != nil {
log.Fatal(err)
```

```
}


// 자동 동기화 재활성화
pinner.SetAutosync(true)
```

콘텐츠 제공자와 함께 사용

```
// 고정된 콘텐츠를 네트워크에 제공하는 설정
pinner, err := dspinner.New(ctx, dstore, dagService,
dspinner.WithPinnedProvider(contentProvider),
dspinner.WithRootsProvider(rootProvider),
)
if err != nil {
log.Fatal(err)
}
```

배치 처리

```
// 대량의 CID를 효율적으로 처리
func batchPin(ctx context.Context, pinner pin.Pinner, cids []cid.Cid) error {
// 자동 동기화 비활성화
oldAutoSync := pinner.SetAutosync(false)
defer pinner.SetAutosync(oldAutoSync)
```

```
for i, c := range cids {
err := pinner.PinWithMode(ctx, c, pin.Direct, fmt.Sprintf("batch-%d", i))
if err != nil {
return fmt.Errorf("CID %s 고정 실패: %w", c, err)
}
// 100개마다 중간 플러시
if (i+1)%100 == 0 {
if err := pinner.Flush(ctx); err != nil {
return fmt.Errorf("중간 플러시 실패: %w", err)
}
}
}
// 최종 플러시
return pinner.Flush(ctx)
}
```

고정 전략과 관리

고정 전략 수립

1. **중요도별 분류**

- 핵심 데이터: 재귀적 고정
- 임시 데이터: 직접 고정
- 캐시 데이터: 고정하지 않음

2. **저장소 관리**

- 정기적인 고정 목록 검토
- 불필요한 고정 해제
- 저장 공간 모니터링

3. **백업 전략**

- 로컬 고정과 원격 고정 병행
- 여러 원격 서비스 활용
- 정기적인 고정 상태 확인

모니터링 및 유지보수

```
// 고정 상태 모니터링 함수
func monitorPins(ctx context.Context, pinner pin.Pinner) {
ticker := time.NewTicker(1 * time.Hour)
defer ticker.Stop()
for {
select {
case <-ticker.C:
// 직접 고정 개수 확인
directCount := 0
directPins := pinner.DirectKeys(ctx, false)
for range directPins {
directCount++
}
```

```
// 재귀적 고정 개수 확인
recursiveCount := 0
recursivePins := pinner.RecursiveKeys(ctx, false)
for range recursivePins {
recursiveCount++
}
log.Printf("고정 상태 - 직접: %d, 재귀적: %d", directCount, recursiveCount)
case <-ctx.Done():
return
}
}
}
```

모범 사례

1. 효율적인 고정 관리

```
// 고정 이름에 메타데이터 포함
pinName := fmt.Sprintf("project:%s:type:%s:date:%s",
projectName, contentType, time.Now().Format("2006-01-02"))

err = pinner.Pin(ctx, node, true, pinName)
```

2. 오류 처리

```
func safePinning(ctx context.Context, pinner pin.Pinner, c cid.Cid, name strin
// 이미 고정되어 있는지 확인
_, pinned, err := pinner.IsPinned(ctx, c)
if err != nil {
return fmt.Errorf("고정 상태 확인 실패: %w", err)
}
if pinned {
log.Printf("CID %s는 이미 고정되어 있습니다", c)
return nil
}
// 고정 시도
err = pinner.PinWithMode(ctx, c, pin.Recursive, name)
if err != nil {
return fmt.Errorf("고정 실패: %w", err)
}
log.Printf("CID %s가 성공적으로 고정되었습니다", c)
return nil
}
```

3. 리소스 관리

```
// 컨텍스트 타임아웃 설정
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

// 고정 작업 수행
err := pinner.Pin(ctx, node, true, "timeout-protected-pin")
if err != nil {
if ctx.Err() == context.DeadlineExceeded {
log.Println("고정 작업이 타임아웃되었습니다")
} else {
log.Printf("고정 오류: %v", err)
}
}
```

문제 해결

일반적인 오류와 해결책

1. **ErrNotPinned**: 고정되지 않은 CID를 해제하려 할 때

```
if errors.Is(err, pin.ErrNotPinned) {
log.Println("이미 고정 해제된 CID입니다")
return nil
```

```
}
```

2. **인덱스 불일치**: 데이터스토어와 인덱스 간 불일치

```
// Pinner 재구축으로 해결
pinner, err := dspinner.New(ctx, dstore, dagService)
```

3. **메모리 부족**: 대량 고정 시 메모리 사용량 증가

```
// 배치 크기 조절 및 중간 플러시
if  i%batchSize == 0 {
pinner.Flush(ctx)
}
```

Pinning은 IPFS 생태계에서 데이터 지속성을 보장하는 핵심 기능입니다. 적절한 고정 전략과 모니터링을 통해 안정적인 데이터 관리를 구현할 수 있습니다.